

云计算技术课程设计报告

课程名称：云计算技术
课程代码：SCAI004712
学号：2023112573
姓名：张春冉
班级：软件工程2023-02班
队友：2023112551 邓苏鑫
完成日期：2026年6月12日

一、华为云环境信息

本课程设计使用华为云华北-北京四区域，Region 为 cn-north-4。实验使用 CCE Standard 集群 cloud-compute-cce，Worker 节点 2 台，Kubernetes 版本为 v1.34.6-r0-34.0.23，满足任务书要求的 1.27 及以上版本。镜像仓库使用 SWR 组织 cloud-compute-2026，主要镜像包括 backend:v1、frontend:v1 和 mpi4py:latest。

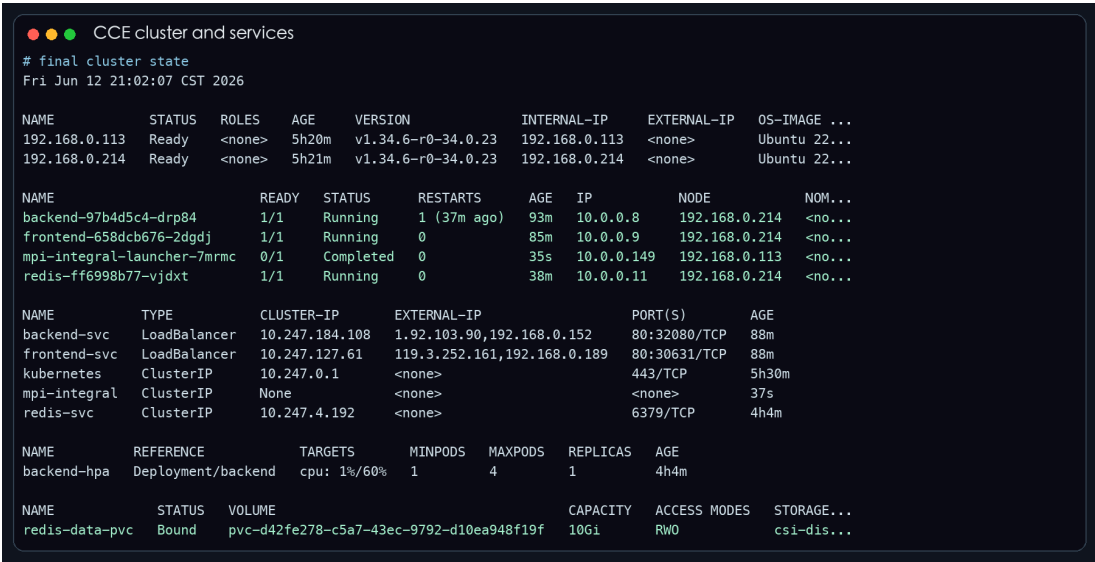


图1 CCE 集群、Service、PVC、HPA 状态

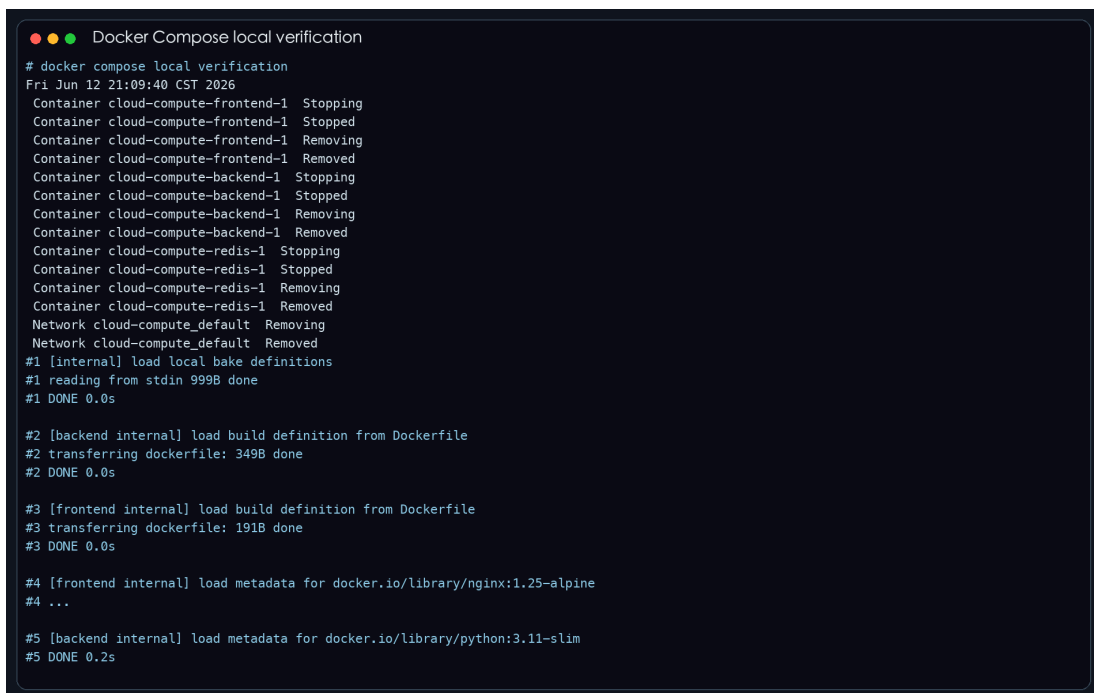
二、第一部分：云计算平台搭建

2.1 应用容器化

本实验实现了 Flask API + Redis + Nginx 静态前端的两层 Web 应用。后端提供 /api/ping、/api/count 和 /api/httpbin 接口，其中 /api/ping 会访问 Redis 并返回 status=ok、学生姓名、时间和 Redis 连接状态。requirements.txt 中除 Flask、Redis 客户端外，额外加入 requests==2.32.3，满足“至少加入 1 个自选 Python 包”的要求。前端首页包含 2023112573 张春冉，可用于验收识别。

后端 Dockerfile 采用多阶段构建，先在 builder 阶段安装依赖，再复制到运行镜像，避免把构建工具带入最终镜像。前端镜像基于 nginx:1.25-alpine，复制 nginx.conf 和静态页面。由于本机为 Apple Silicon，而 CCE Worker 为 amd64，推送 SWR 时使用 docker buildx build --platform linux/amd64 --provenance=false --sbom=false --push，保证镜像架构正确，同时避免 SWR 对 BuildKit provenance manifest 的解析问题。

本地联调用 Docker Compose 启动 backend、frontend、redis 三个容器。宿主机 5000 端口被占用，因此后端映射为 5001:5000，容器内部端口和前端反向代理仍然保持 5000。联调结果显示前端 Nginx 访问 /api/ping 能返回 Redis connected，后端日志中也出现对应 GET 请求。



```
Docker Compose local verification
# docker compose local verification
Fri Jun 12 21:09:40 CST 2026
Container cloud-compute-frontend-1 Stopping
Container cloud-compute-frontend-1 Stopped
Container cloud-compute-frontend-1 Removing
Container cloud-compute-frontend-1 Removed
Container cloud-compute-backend-1 Stopping
Container cloud-compute-backend-1 Stopped
Container cloud-compute-backend-1 Removing
Container cloud-compute-backend-1 Removed
Container cloud-compute-redis-1 Stopping
Container cloud-compute-redis-1 Stopped
Container cloud-compute-redis-1 Removing
Container cloud-compute-redis-1 Removed
Network cloud-compute_default Removing
Network cloud-compute_default Removed
#1 [internal] load local bake definitions
#1 reading from stdin 999B done
#1 DONE 0.0s

#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 349B done
#2 DONE 0.0s

#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 191B done
#3 DONE 0.0s

#4 [frontend internal] load metadata for docker.io/library/nginx:1.25-alpine
#4 ...

#5 [backend internal] load metadata for docker.io/library/python:3.11-slim
#5 DONE 0.2s
```

图2 Docker Compose 本地联调

```
● ● ● MPI image build and push
# MPI image rebuild with ssh fix, single manifest push
#1 DONE 0.0s
#2 DONE 0.3s
#3 DONE 0.0s
#4 DONE 0.0s
#5 [1/4] FROM docker.io/library/python:3.12-slim@sha256:401f6e1a67dad31a1bd78e9ad22d0ee0a3b52154e6bd30...
#5 resolve docker.io/library/python:3.12-slim@sha256:401f6e1a67dad31a1bd78e9ad22d0ee0a3b52154e6bd30e90...
#5 DONE 0.0s
#9 exporting manifest sha256:da058f01218928285ed16f54a9fde94f35e79f037b19ccd12480ae898240c3ba done
#9 exporting config sha256:aa64e7c0848fe638f56e4a7c98d255778b40484b9acdda7c3799d9d31e466108 done
#9 naming to swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/mpi4py:latest done
#10 DONE 0.0s
#9 pushing manifest for swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/mpi4py:latest@sha256:da058...
#9 pushing manifest for swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/mpi4py:latest@sha256:da058...
#9 DONE 1.7s
```

图3 MPI 镜像构建与推送到 SWR

2.2 CCE 集群搭建

CCE 集群已经通过本地 kubeconfig 连接验证。kubectl get nodes -o wide 输出中两个 Worker 节点均为 Ready，版本列显示 v1.34.6-r0-34.0.23。这说明集群版本、节点数量和节点状态都满足任务书要求。

2.3 应用部署

K8s 资源文件位于 k8s/ 目录。后端 Deployment 设置 replicas: 2，包含 CPU/内存 requests 与 limits，通过 envFrom 引用 backend-config 中的 Redis 地址，通过 secretKeyRef 引用 Redis 密码。Redis Deployment 设置 1 个副本，内存 limit 为 512Mi，并挂载 PVC。后端 Service 为 LoadBalancer，带有华为云 ELB 自动创建注解；Redis Service 为 ClusterIP，仅供集群内部访问。前端也通过 LoadBalancer 暴露公网访问入口。

实际部署中遇到过两个问题：第一，Pod 拉取 SWR 镜像时最初出现认证问题，解决办法是在 Deployment 中加入 imagePullSecrets: default-secret；第二，LoadBalancer Service 曾处于 pending，原因是缺少华为云 ELB 创建参数，补充 kubernetes.io/elb.autocreate 后公网 IP 正常生成。当前后端公网 IP 为 1.92.103.90，前端公网 IP 为 119.3.252.161。



图4 前端页面访问和 API 返回

2.4 持久化存储

Redis 使用 PVC redis-data-pvc, 存储类为 csi-disk, 容量 10Gi, 状态为 Bound。验证步骤为：先在 Redis 中写入 testkey=hello, 再删除当前 Redis Pod, 等待 Deployment 自动创建新 Pod, 最后重新读取 testkey。重建后仍然返回 hello, 说明 Redis 的 /data 目录已经由 PVC 持久化, 不依赖单个 Pod 生命周期。

```
Redis PVC persistence
## pvc before
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS
redis-data-pvc Bound     pvc-d42fe278-c5a7-43ec-9792-d10ea948f19f 10Gi        RWO            csi-disk
## read testkey before delete
hello
pod/redis-ff6998b77-vjdxr condition met
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE           NOMINATED NODE   ...
redis-ff6998b77-vjdxr 1/1     Running   0          45s   10.0.0.11    192.168.0.214 <none>          ...
## read testkey after recreate
hello
```

图5 Redis PVC 持久化验证

2.5 ConfigMap Volume 挂载

Nginx 反向代理配置保存在 ConfigMap frontend-nginx-conf 中, 并通过 Volume 挂载到 /etc/nginx/conf.d/default.conf。实际文件中可看到: proxy_pass http://backend-svc:80/api/;, 说明前端容器读取的是 ConfigMap 文件内容, 而不是镜像内固定配置。

Volume 挂载适合 Nginx 配置、证书、应用配置文件等以文件形式存在的配置；envFrom 更适合主机名、端口、环境标识这类键值型配置。本实验中后端用 envFrom 读取 Redis 地址和端口，前端用 ConfigMap Volume 挂载 Nginx 配置，两者用途不同。

```
ConfigMap volume mount
## ConfigMap volume verification
default.conf: |
    root /usr/share/nginx/html;
    proxy_pass http://backend-svc:80/api;
kind: ConfigMap
  {"apiVersion":"v1","data":{"default.conf":"server {\n    listen 80;\n    server_name _;\n    r...
  name: frontend-nginx-conf
## mounted /etc/nginx/conf.d/default.conf
    root /usr/share/nginx/html;
    proxy_pass http://backend-svc:80/api;
```

图6 ConfigMap Volume 挂载验证

2.6 HPA 弹性伸缩

后端 Deployment 配置 HPA, minReplicas=1, maxReplicas=4, 目标 CPU 利用率 60%。集群最初没有 Metrics API, 因此先安装 metrics-server, 并将镜像改为华为云可拉取的镜像地址。kubectl top nodes、kubectl top pods 有数据后, 用 CPU 压力和 ab 请求压测后端 API。压测期间 HPA 监测到 CPU 利用率高于目标值, 副本数从 1 增加到 4; 停止压测并等待稳定窗口后, 多余 Pod 进入 Terminating, 最终缩回 1 个副本。

```
HPA scale up
# HPA scale up: CPU exceeds target, replicas grow to 4
## sample 2
NAME      REFERENCE      TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 244%/60%  1         4         1         3h29m
NAME      READY   STATUS    RESTARTS   AGE
backend-97b4d5c4-cctxc  0/1     ContainerCreating  0         1s
backend-97b4d5c4-drp84  1/1     Running        1 (2m5s ago)  58m
backend-97b4d5c4-v9kxn  1/1     Running        0         1s
backend-97b4d5c4-xfj6l  0/1     ContainerCreating  0         1s
NAME      CPU(cores)  MEMORY(bytes)
## sample 3
NAME      REFERENCE      TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 499%/60%  1         4         4         3h30m
NAME      READY   STATUS    RESTARTS   AGE
backend-97b4d5c4-cctxc  1/1     Running        0         22s
backend-97b4d5c4-drp84  1/1     Running        1 (2m26s ago)  58m
backend-97b4d5c4-v9kxn  1/1     Running        0         22s
backend-97b4d5c4-xfj6l  1/1     Running        0         22s
NAME      CPU(cores)  MEMORY(bytes)
```

图7 HPA 扩容过程

```
●●● HPA scale down
# HPA scale down: CPU becomes low, extra pods terminate
## sample 11
NAME      REFERENCE    TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa Deployment/backend  cpu: 1%/60%  1         4         4         3h38m
NAME      READY    STATUS    RESTARTS  AGE
backend-97b4d5c4-cctxc  1/1      Running   0         8m53s
backend-97b4d5c4-drp84  1/1      Running   1 (10m ago)  67m
backend-97b4d5c4-v9kxn  1/1      Running   0         8m53s
backend-97b4d5c4-xfj6l  1/1      Running   0         8m53s
NAME      CPU(cores)  MEMORY(bytes)
## sample 12
NAME      REFERENCE    TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa Deployment/backend  cpu: 1%/60%  1         4         4         3h39m
NAME      READY    STATUS    RESTARTS  AGE
backend-97b4d5c4-cctxc  1/1      Terminating  0         9m24s
backend-97b4d5c4-drp84  1/1      Running      1 (11m ago)  68m
backend-97b4d5c4-v9kxn  1/1      Terminating  0         9m24s
backend-97b4d5c4-xfj6l  1/1      Terminating  0         9m24s
NAME      CPU(cores)  MEMORY(bytes)
```

图8 HPA 缩容过程

HPA 扩容存在延迟，主要原因是 metrics-server 有采集周期，HPA 控制器也按固定间隔评估指标。缩容更慢，是因为 Kubernetes 需要稳定窗口避免短时间流量波动造成副本频繁增删。对于云平台，HPA 的价值在于低负载时减少副本占用，高负载时自动补充处理能力，从而兼顾成本和可用性。

三、第二部分：MPI 并行科学计算

3.1 MPI 环境部署

本项目选择方向 B：MPI 并行科学计算。已经在 CCE 中部署 MPI Operator，并使用 MPIJob 提交 mpi4py 作业。MPI 镜像基于 python:3.12-slim，安装 OpenMPI、mpi4py 和 openssh-server。由于 MPI Operator 通过 SSH 启动 worker 进程，镜像中开启 PermitRootLogin yes、PubkeyAuthentication yes，并设置 StrictModes no，避免 Secret 挂载的 SSH key 权限被 sshd 拒绝。

mpi/mpijob.yaml 使用 kubeflow.org/v2beta1，设置 slotsPerWorker: 2、Worker 副本数 2。开始时 Launcher 曾因 hostfile 参数和 DNS 时序失败，最终通过显式指定 --hostfile /etc/mpi/hostfile、--mca plm_rsh_args，并设置 launcherCreationPolicy: WaitForWorkersReady 解决。最终日志输出 mpi_blocking p=4 n=10000000 pi=3.1415926536，MPIJob 状态为 Succeeded。

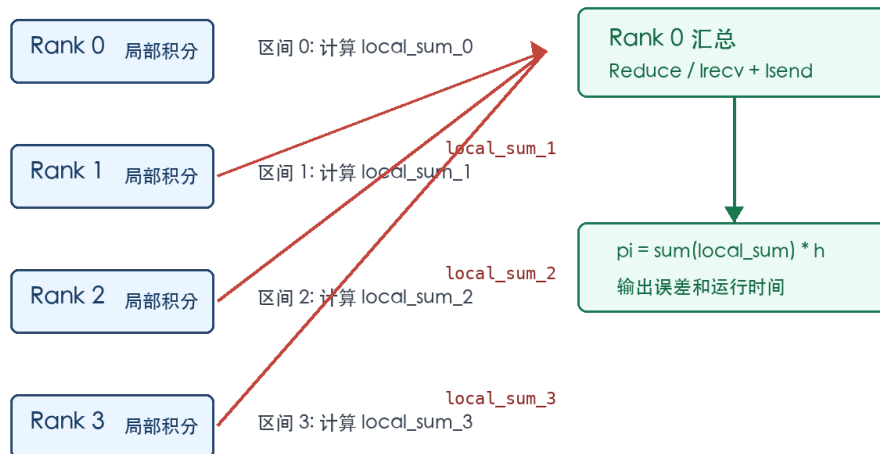
```
●●● MPIJob cloud result
# MPIJob cloud run after WaitForWorkersReady
mpijob.kubeflow.org "mpi-integral" deleted
mpijob.kubeflow.org/mpi-integral created
--- sample 1 21:01:31 ---
mpi-integral-worker-0 0/1 ContainerCreating 0 1s <none> 192.168.0.214 <none> ...
mpi-integral-worker-1 0/1 ContainerCreating 0 1s <none> 192.168.0.113 <none> ...
--- sample 2 21:01:37 ---
mpi-integral-launcher-7mrmc 1/1 Running 0 5s 10.0.0.149 192.168.0.113 <none> ...
mpi-integral-worker-0 1/1 Running 0 7s 10.0.0.19 192.168.0.214 <none> ...
mpi-integral-worker-1 1/1 Running 0 7s 10.0.0.148 192.168.0.113 <none> ...
Created Running {"Launcher":{"active":1},"Worker":{"active":2}}
--- sample 3 21:01:43 ---
mpi-integral-launcher-7mrmc 0/1 Completed 0 11s 10.0.0.149 192.168.0.113 <none>...
Created Running Succeeded {"Launcher":{"succeeded":1},"Worker":{"}}
Warning: Permanently added 'mpi-integral-worker-1.mpi-integral.default.svc' (ED25519) to the list of k...
Warning: Permanently added 'mpi-integral-worker-0.mpi-integral.default.svc' (ED25519) to the list of k...
mpi_blocking p=4 n=10000000 pi=3.1415926536 time=4.223537s error=1.234568e-13
Name: mpi-integral
Launcher Creation Policy: WaitForWorkersReady
Clean Pod Policy: Running
Message: MPIJob default/mpi-integral is created.
Message: MPIJob default/mpi-integral is running.
Reason: MPIJobRunning
Type: Running
Message: MPIJob default/mpi-integral successfully completed.
Reason: MPIJobSucceeded
Type: Succeeded
Succeeded: 1
Normal MPIJobCreated 14s mpi-job-controller MPIJob default/mpi-integral is created.
Normal MPIJobRunning 5s (x4 over 11s) mpi-job-controller MPIJob default/mpi-integral is running
Normal MPIJobSucceeded 4s mpi-job-controller MPIJob default/mpi-integral successfu...
```

图9 MPIJob 云端运行结果

3.2 并行算法实现

算法选择数值积分。串行版使用中点公式计算函数 $4/(1+x*x)$ 在 $[0,1]$ 上的积分，结果近似 π 。MPI 阻塞版中，rank 0 根据进程数切分区间，并通过 `comm.scatter` 把每个子区间发送给对应进程；每个进程计算自己的 `local_sum` 后，通过 `comm.reduce(local_sum, op=MPI.SUM, root=0)` 汇总到 rank 0。非阻塞版同样使用 `scatter` 分发区间，汇总阶段改为 rank 0 先 `Irecv`，其他进程 `Isend`，最后 `Waitall` 后累加结果。

数值积分 MPI 通信模式示意图



阻塞版使用 Reduce；非阻塞版中 Rank 0 先 lrecv，其他 Rank lsend，最后 Waitall 汇总。

图10 MPI 通信模式示意图

阻塞版核心代码如下：

```

import math
import sys
import time

from mpi4py import MPI

def f(x):
    return 4.0 / (1.0 + x * x)

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

n = int(sys.argv[1]) if len(sys.argv) > 1 else 10_000_000
h = 1.0 / n

if rank == 0:
    chunk = n // size
    ranges = []
    for worker in range(size):
        start_i = worker * chunk
        end_i = n if worker == size - 1 else start_i + chunk
        ranges.append((start_i, end_i))
else:
    ranges = None

# Scatter: rank 0 sends one integration interval range to each MPI process.
start_i, end_i = comm.scatter(ranges, root=0)

start = time.perf_counter()

```



```

local_sum = 0.0
for i in range(start_i, end_i):
    x = (i + 0.5) * h
    local_sum += f(x)

# Reduce: every process sends local_sum to rank 0, rank 0 sums all partial areas.
total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

if rank == 0:
    pi = total_sum * h
    elapsed = time.perf_counter() - start
    print(f"mpi_blocking p={size} n={n} pi={pi:.10f} time={elapsed:.6f}s error=
          {abs(math.pi - pi):.6e}")

```

非阻塞版核心代码如下：

```

import math
import sys
import time
from array import array

from mpi4py import MPI

def f(x):
    return 4.0 / (1.0 + x * x)

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

n = int(sys.argv[1]) if len(sys.argv) > 1 else 10_000_000
h = 1.0 / n

if rank == 0:
    chunk = n // size
    ranges = []
    for worker in range(size):
        start_i = worker * chunk
        end_i = n if worker == size - 1 else start_i + chunk
        ranges.append((start_i, end_i))
else:
    ranges = None

# Scatter: rank 0 sends one integration interval range to each MPI process.
start_i, end_i = comm.scatter(ranges, root=0)

start = time.perf_counter()
local_sum = 0.0
for i in range(start_i, end_i):
    x = (i + 0.5) * h
    local_sum += f(x)

if rank == 0:
    total_sum = local_sum
    requests = []

```

```

buffers = []
for source in range(1, size):
    buf = array("d", [0.0])
    # Irecv: rank 0 posts receives first, so worker Isend calls can complete
    # asynchronously.
    requests.append(comm.Irecv(buf, source=source, tag=100 + source))
    buffers.append(buf)
MPI.Request.Waitall(requests)
for buf in buffers:
    total_sum += buf[0]
pi = total_sum * h
elapsed = time.perf_counter() - start
print(f"mpi_nonblocking p={size} n={n} pi={pi:.10f} time={elapsed:.6f}s error=
      {abs(math.pi - pi):.6e}")
else:
    send_buf = array("d", [local_sum])
    # Isend: worker sends its local partial sum to rank 0 without blocking on
    # immediate receive completion.
    req = comm.Isend(send_buf, dest=0, tag=100 + rank)
    req.Wait()

```

3.3 性能测试与 Amdahl 分析

固定积分点数为 10,000,000，在 CCE 上分别使用 1、2、4 个 MPI 进程运行，每种进程数运行 3 次取平均。测试结果如下。

进程数 p	三次运行 时间/s	平均 T(p)/s	实测加速比 S	Amdahl 理论值
1	6.079585, 5.597226, 6.064438	5.913750	1.00	1.00
2	5.785596, 6.145806, 6.287902	6.073101	0.97	1.26
4	4.100012, 4.199975, 3.999982	4.099990	1.44	1.44

估算可并行比例 $f = 0.409$ 。4 进程阻塞平均 4.099990s，非阻塞平均 3.709143s。

```
●●● MPI performance on CCE

# MPI performance summary on CCE
| p | T(p)/s | S | Amdahl | |
|---|---|---|---|---|
| 1 | 6.079585, 5.597226, 6.064438 | 5.913750 | 1.00 | 1.00 |
| 2 | 5.785596, 6.145806, 6.287902 | 6.073101 | 0.97 | 1.26 |
| 4 | 4.100012, 4.199975, 3.999982 | 4.099990 | 1.44 | 1.44 |

f = 0.409
4.099990s 3.709143s

mpi_blocking p=1 n=10000000 pi=3.1415926536 time=6.079585s error=6.217249e-14
Created Running Succeeded
mpi_blocking p=1 n=10000000 pi=3.1415926536 time=5.597226s error=6.217249e-14
Created Running Succeeded
mpi_blocking p=1 n=10000000 pi=3.1415926536 time=6.064438s error=6.217249e-14
Created Running Succeeded
mpi_blocking p=2 n=10000000 pi=3.1415926536 time=5.785596s error=1.296740e-13
Created Running Succeeded
mpi_blocking p=2 n=10000000 pi=3.1415926536 time=6.145806s error=1.296740e-13
Created Running Succeeded
mpi_blocking p=2 n=10000000 pi=3.1415926536 time=6.287902s error=1.296740e-13
Created Running Succeeded
mpi_blocking p=4 n=10000000 pi=3.1415926536 time=4.100012s error=1.234568e-13
Created Running Succeeded
mpi_blocking p=4 n=10000000 pi=3.1415926536 time=4.199975s error=1.234568e-13
Created Running Succeeded
mpi_blocking p=4 n=10000000 pi=3.1415926536 time=3.999982s error=1.234568e-13
mpi_nonblocking p=4 n=10000000 pi=3.1415926536 time=3.727457s error=1.234568e-13
mpi_nonblocking p=4 n=10000000 pi=3.1415926536 time=3.799980s error=1.234568e-13
Created Running Succeeded
mpi_nonblocking p=4 n=10000000 pi=3.1415926536 time=3.599992s error=1.234568e-13
```

图11 MPI 性能测试汇总

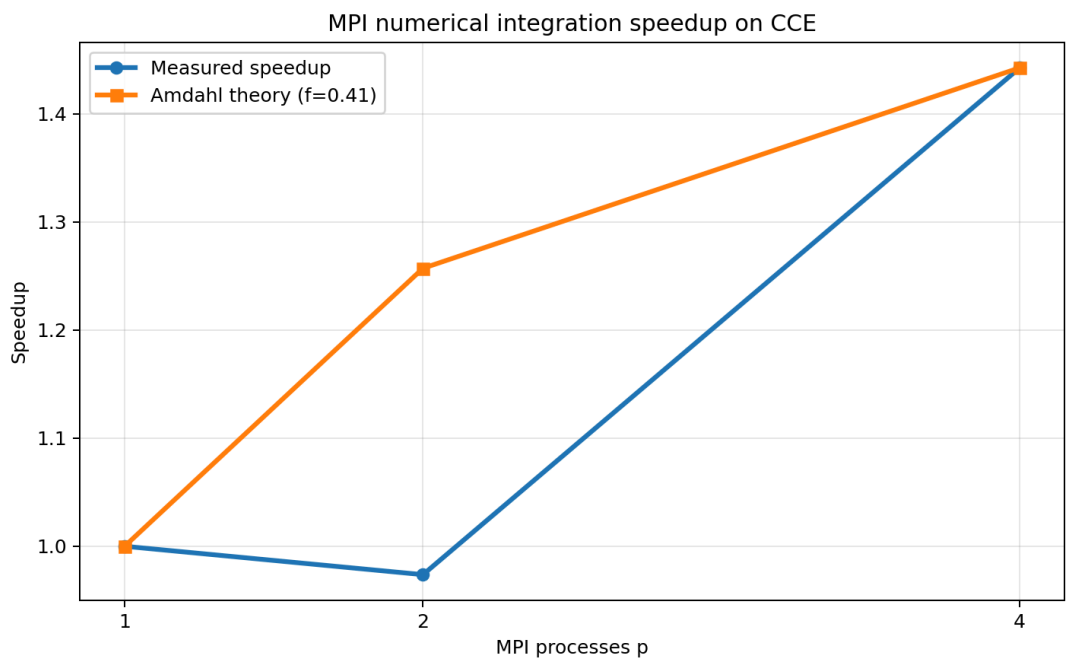


图12 实测加速比与 Amdahl 理论加速比

从结果看，2 进程平均时间反而略高于 1 进程，4 进程才体现出加速。原因是本实验中积分计算量虽然可并行，但每个 MPIJob 都运行在容器和 Kubernetes 调度环境中，进程启动、SSH 建连、跨节点通信、Reduce 同步都带来额外开销。2 进程时计算节省不足以抵消这些开销；4 进程时局部循环计算减少更多，才获得约 1.44 倍加速。按 4 进程实测加速比反推，可并行比例约为 0.409。这个数值不代表算法理论上只有 40.9% 可并行，而是把云端调度和通信开销也折算进了实际系统表现。

3.4 非阻塞通信优化

非阻塞版本在 4 进程下平均时间为 3.709143s，阻塞版本为 4.099990s，本次实验中非阻塞版本略快。原因是 rank 0 提前投递多个 Irecv，worker 的 Isend 可以更早完成发送请求，减少最后汇总阶段的等待。由于本题每个进程只发送一个 double 类型局部和，通信量很小，所以改善幅度有限。如果换成矩阵分块、排序相邻交换或边界数据较大的科学计算，非阻塞通信更容易体现计算与通信重叠的价值。

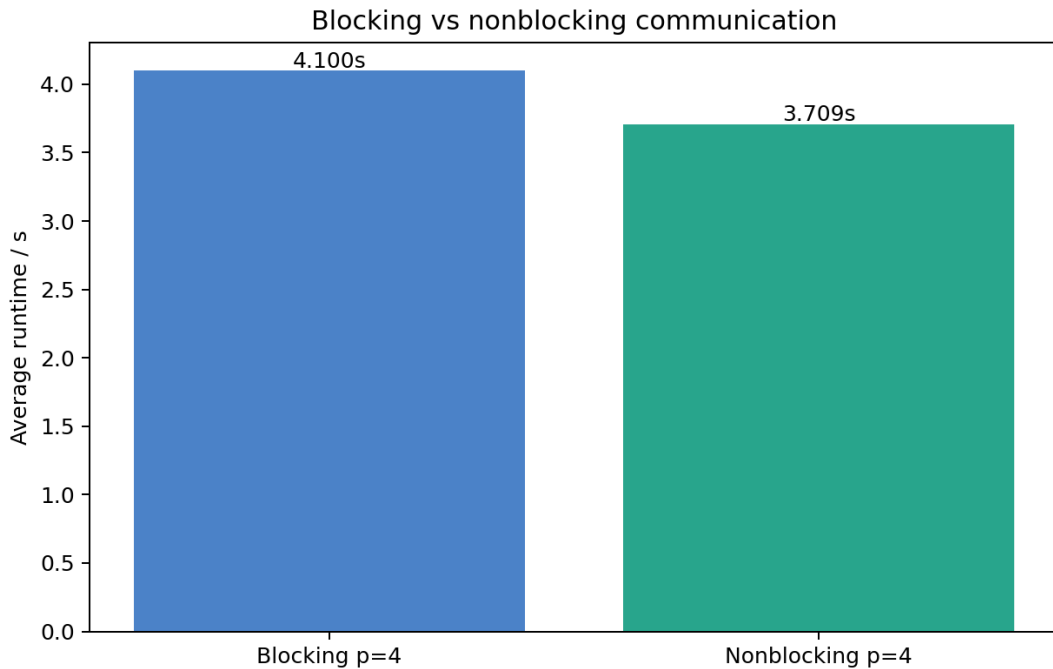


图13 阻塞通信与非阻塞通信对比

四、附加题

4.1 监控系统

附加题 1 部署了轻量 Prometheus + Grafana。Prometheus 通过 Kubernetes service discovery 发现节点和 Pod，并从 kubelet/cAdvisor 拉取容器 CPU、内存等指标。Grafana 通过 provisioning 自动配置 Prometheus datasource 和 dashboard。Dashboard 中包含节点 CPU 使用折线图和各 Pod 内存使用柱状图，能看到 default、monitoring、kube-system、mpi-operator 等命名空间下的 Pod 指标。

```

Prometheus and Grafana deploy

# monitoring deploy status
grafana-c9dfbd6c8-f5gnj 0/1 ContainerCreating 0 14s <none> 192.168.0.113 <none> <non
prometheus-c4c45cd87-f54bq 0/1 ContainerCreating 0 15s <none> 192.168.0.214 <none> <non
grafana-c9dfbd6c8-f5gnj 1/1 Running 0 25s 10.0.0.200 192.168.0.113 <none> <none>
prometheus-c4c45cd87-f54bq 1/1 Running 0 26s 10.0.0.45 192.168.0.214 <none> <none>
grafana-c9dfbd6c8-f5gnj 1/1 Running 0 35s 10.0.0.200 192.168.0.113 <none> <none>
prometheus-c4c45cd87-f54bq 1/1 Running 0 36s 10.0.0.45 192.168.0.214 <none> <none>
grafana-c9dfbd6c8-f5gnj 1/1 Running 0 45s 10.0.0.200 192.168.0.113 <none> <none>
prometheus-c4c45cd87-f54bq 1/1 Running 0 46s 10.0.0.45 192.168.0.214 <none> <none>
grafana-c9dfbd6c8-f5gnj 1/1 Running 0 56s 10.0.0.200 192.168.0.113 <none> <none>
prometheus-c4c45cd87-f54bq 1/1 Running 0 57s 10.0.0.45 192.168.0.214 <none> <none>
grafana-c9dfbd6c8-f5gnj 1/1 Running 0 66s 10.0.0.200 192.168.0.113 <none> <none>
prometheus-c4c45cd87-f54bq 1/1 Running 0 67s 10.0.0.45 192.168.0.214 <none> <none>
grafana-c9dfbd6c8-f5gnj 1/1 Running 0 77s 10.0.0.200 192.168.0.113 <none> <none>
prometheus-c4c45cd87-f54bq 1/1 Running 0 78s 10.0.0.45 192.168.0.214 <none> <none>

--- services ---
grafana ClusterIP 10.247.31.63 <none> 3000/TCP 99s
prometheus ClusterIP 10.247.177.79 <none> 9090/TCP 100s

--- grafana/provisioning configmaps ---
grafana-provisioning 2 100s
grafana-dashboard 1 100s
prometheus-config 1 101s

--- prometheus targets local query via port-forward will be captured separately ---

```

图14 Prometheus 与 Grafana 部署状态

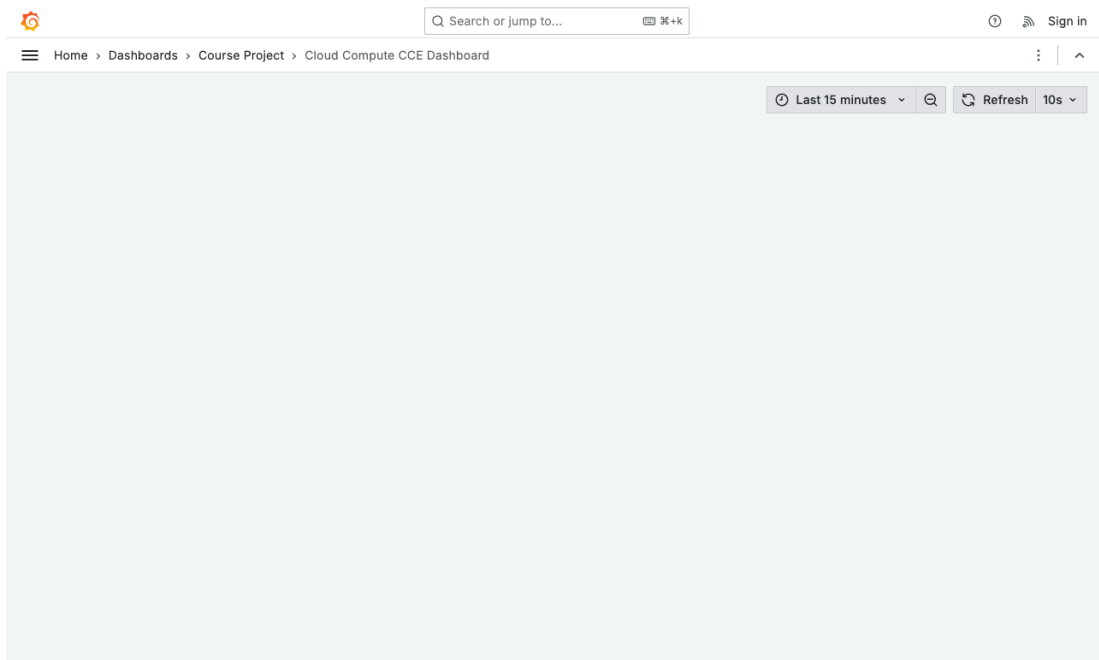


图15 Grafana Dashboard

Prometheus 的 Pull 模式是由服务端周期性向目标 endpoint 发起 HTTP 请求并抓取 metrics，这种方式便于集中配置抓取周期、标签和告警规则。Dashboard 中使用的三个主要指标含义为：container_cpu_usage_seconds_total 表示容器累计 CPU 使用时间，通常配合 rate() 观察单位时间内 CPU 消耗；container_memory_working_set_bytes 表示容器当前活跃内存工作集，更接近实际内存占用；up 表示目标是否可被 Prometheus 成功抓取，值为 1 表示当前目标在线。

4.2 CI/CD 流水线

附加题 2 编写并实际运行了 `.github/workflows/deploy-cce.yml`。流水线在 push 或手动触发后执行：Checkout 源码、生成镜像 Tag、登录华为云 SWR、构建并推送 backend/frontend 镜像、写入 kubeconfig、执行 `kubectl set image` 更新 CCE Deployment，最后等待 rollout 完成并输出 Deployment 状态。敏感信息全部通过 GitHub Secrets 提供，包括 `SWR_USERNAME`、`SWR_PASSWORD` 和 `CCE_KUBECONFIG_B64`。本项目远程仓库地址为 `https://github.com/ln-one/Cloud-Compute-Course-Project`，用于部署验证的流水线 Run ID 为 27419440447，提交短哈希为 24dbcce，运行结论为 success。

```
CI/CD workflow verification
# CI/CD workflow verification
name: Build and deploy to Huawei CCE
jobs:
  - name: Checkout source
  - name: Set image tag
  - name: Login to Huawei SWR
    username: ${ secrets.SWR_USERNAME }}
  - name: Build and push backend image
  - name: Build and push frontend image
  - name: Configure kubectl
  - name: Update CCE deployments
workflow name: Build and deploy to Huawei CCE
jobs: build-and-deploy
step count: 7
deploy step: Update CCE deployments
```

图16 CI/CD 工作流文件校验

图17 GitHub Actions CI/CD 实际运行成功

Repository: ln-one/Cloud-Compute-Course-Project
Workflow: Build and deploy to Huawei CCE
Run ID: 27419440447
Branch / Commit: main / 24dbcce
Conclusion: success
Job: build-and-deploy · success · 2026-06-12T13:41:32Z – 2026-06-12T13:42:29Z
Steps: Checkout -> Login SWR -> Build backend -> Build frontend -> Configure kubectl -> Update CCE
URL: <https://github.com/ln-one/Cloud-Compute-Course-Project/actions/runs/27419440447>

图17 GitHub Actions CI/CD 实际运行成功

图18 GitHub Actions 后 CCE Deployment 镜像更新

CCE deployment images after GitHub Actions
2026-06-12 21:43:14 CST
NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS IMAGES SELECTOR
backend 1/1 1 1 4h45m backend swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/backend:24dbcce app=backend
frontend 1/1 1 1 4h45m frontend swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/frontend:24dbcce app=frontend
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
backend-bd6f4d844-qxf9d 1/1 Running 0 54s 10.0.0.47 192.168.0.214 <none> <none>
frontend-5889d5b768-9wq44 1/1 Running 0 53s 10.0.0.48 192.168.0.214 <none> <none>
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
backend-hpa Deployment/backend cpu: 1%/60% 1 4 1 4h45m

图18 GitHub Actions 后 CCE Deployment 镜像更新

持续集成强调每次代码提交后自动构建、测试和发现问题；持续部署是在集成通过后自动把新版本发布到目标环境。本项目的工作流把构建镜像和更新 CCE Deployment 串起来，属于端到端部署流水线。GitOps 的核心理念是把期望状态写入 Git 仓库，集群实际状态由自动化系统持续向 Git 中的声明式配置收敛。本次实验中，push 到 main 分支后，GitHub Actions 自动构建两个镜像并推送到 SWR，然后把 CCE 中 backend 与 frontend 的镜像更新为 24dbcce 标签，Deployment 均完成 rollout。

4.3 前沿专题：边缘计算模拟 K3s + MQTT

附加题 3 选择 C-2。实验用本地 Docker 运行 Mosquitto 模拟边缘节点上的 MQTT Broker，用 sensor_publisher.py 模拟边缘传感器，每 0.5 秒发布一次温度、湿度和时间戳。cloud_mqtt_bridge.py 作为云端桥接服务订阅 edge/sensor/# 主题，并通过 kubectl port-forward svc/redis-svc 63790:6379 将数据写入 CCE 集群中的 Redis。Redis 中保存两个结构：edge:sensor:latest 列表保存最近 20 条消息，edge:sensor:last 哈希保存最后一条消息。

```
Edge MQTT to CCE Redis
# edge MQTT to CCE Redis verification
--- broker ---
edge-mosquitto Up 28 seconds 0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 1, 'temperature': 25.76, 'humidity': 46.51, 'timestamp': '2026-06-12T13:28:29.0808'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 2, 'temperature': 24.45, 'humidity': 51.8, 'timestamp': '2026-06-12T13:28:29.582'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 3, 'temperature': 26.05, 'humidity': 50.27, 'timestamp': '2026-06-12T13:28:30.0808'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 4, 'temperature': 26.73, 'humidity': 45.74, 'timestamp': '2026-06-12T13:28:30.582'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 5, 'temperature': 25.76, 'humidity': 51.25, 'timestamp': '2026-06-12T13:28:31.0808'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 6, 'temperature': 24.95, 'humidity': 47.5, 'timestamp': '2026-06-12T13:28:31.582'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 7, 'temperature': 25.8, 'humidity': 53.94, 'timestamp': '2026-06-12T13:28:32.0808'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 8, 'temperature': 24.4, 'humidity': 46.98, 'timestamp': '2026-06-12T13:28:32.582'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 9, 'temperature': 24.24, 'humidity': 47.39, 'timestamp': '2026-06-12T13:28:33.0808'}
published edge/sensor/room-a {'device_id': 'edge-room-a-01', 'seq': 10, 'temperature': 25.21, 'humidity': 46.76, 'timestamp': '2026-06-12T13:28:33.582'}
--- redis latest list from CCE redis pod ---
{"device_id": "edge-room-a-01", "seq": 10, "temperature": 25.21, "humidity": 46.76, "timestamp": "2026-06-12T13:28:33.582"}
{"device_id": "edge-room-a-01", "seq": 9, "temperature": 24.24, "humidity": 47.39, "timestamp": "2026-06-12T13:28:33.0808"}
{"device_id": "edge-room-a-01", "seq": 8, "temperature": 24.4, "humidity": 46.98, "timestamp": "2026-06-12T13:28:32.582"}
{"device_id": "edge-room-a-01", "seq": 7, "temperature": 25.8, "humidity": 53.94, "timestamp": "2026-06-12T13:28:32.0808"}
{"device_id": "edge-room-a-01", "seq": 6, "temperature": 24.95, "humidity": 47.5, "timestamp": "2026-06-12T13:28:31.582"}
seq
temperature
humidity
--- bridge log ---
connected mqtt reason=Success
stored topic=edge/sensor/room-a seq=1
stored topic=edge/sensor/room-a seq=2
stored topic=edge/sensor/room-a seq=3
stored topic=edge/sensor/room-a seq=4
stored topic=edge/sensor/room-a seq=5
stored topic=edge/sensor/room-a seq=6
stored topic=edge/sensor/room-a seq=7
stored topic=edge/sensor/room-a seq=8
stored topic=edge/sensor/room-a seq=9
stored topic=edge/sensor/room-a seq=10
```

图19 边缘 MQTT 数据写入 CCE Redis

MQTT 适合边缘场景，主要原因是协议轻量、报文头小、发布订阅模型能降低设备与云端之间的耦合。传感器只需要把数据发布到主题，不需要知道云端 Redis、数据库或业务服务的位置。弱网环境下可通过 QoS 1 保证消息至少送达一次，但也要注意重复消息、离线缓存大小和网络恢复后的突发流量。本实验使用本地 Broker 模拟边缘节点，如果部

署到真实 K3s，可以把 Mosquitto 作为边缘集群中的 Deployment，把桥接程序部署到云端 K8s。云边协同的主要挑战在于链路延迟、网络抖动、边缘节点资源有限以及安全认证。实际系统中应为 MQTT 开启用户名密码或证书认证，并在桥接端做去重和限流。

五、主要代码摘要

后端 Flask 代码摘要：

```
import os
from datetime import datetime, timezone

import redis
import requests
from flask import Flask, jsonify

app = Flask(__name__)

def redis_client():
    host = os.getenv("REDIS_HOST", "localhost")
    port = int(os.getenv("REDIS_PORT", "6379"))
    password = os.getenv("REDIS_PASSWORD") or None
    return redis.Redis(host=host, port=port, password=password,
                       decode_responses=True)

@app.get("/api/ping")
def ping():
    payload = {
        "status": "ok",
        "student": "2023112573 张春冉",
        "time": datetime.now(timezone.utc).isoformat(),
    }
    try:
        redis_client().incr("ping_count")
        payload["redis"] = "connected"
    except Exception as exc:
        payload["redis"] = "unavailable"
        payload["redis_error"] = exc.__class__.__name__
    return jsonify(payload)

@app.get("/api/count")
def count():
    value = redis_client().get("ping_count") or "0"
    return jsonify({"ping_count": int(value)})

@app.get("/api/httpbin")
def httpbin():
    response = requests.get("https://httpbin.org/status/204", timeout=3)
    return jsonify({"status_code": response.status_code})
```



```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

后端 Dockerfile:

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt --target /build/packages

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /build/packages /app/packages
COPY . .
ENV PYTHONPATH=/app/packages
EXPOSE 5000
CMD ["python", "app.py"]
```

前端 Dockerfile:

```
FROM nginx:1.25-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY static/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

MPIJob 摘要:

```
apiVersion: kubeflow.org/v2beta1
kind: MPIJob
metadata:
  name: mpi-integral
  namespace: default
spec:
  launcherCreationPolicy: WaitForWorkersReady
  slotsPerWorker: 2
  runPolicy:
    cleanPodPolicy: Running
  mpiReplicaSpecs:
    Launcher:
      replicas: 1
      template:
        spec:
          imagePullSecrets:
            - name: default-secret
          containers:
            - name: launcher
```

```

        image: swr.cn-north-4.myhuaweicloud.com/cloud-compute-
2026/mpi4py:latest
        env:
          - name: OMPI_ALLOW_RUN_AS_ROOT
            value: "1"
          - name: OMPI_ALLOW_RUN_AS_ROOT_CONFIRM
            value: "1"
        command:
          - mpirun
          - --hostfile
          - /etc/mpi/hostfile
          - --map-by
          - slot
          - --mca
          - plm_rsh_args
          - "-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
ConnectionAttempts=10"
          - -n
          - "4"
          - python
          - /opt/mpi/integral_mpi.py
          - "10000000"
Worker:
  replicas: 2
  template:
    spec:
      imagePullSecrets:
        - name: default-secret
      containers:
        - name: worker
          image: swr.cn-north-4.myhuaweicloud.com/cloud-compute-
2026/mpi4py:latest
          env:
            - name: OMPI_ALLOW_RUN_AS_ROOT
              value: "1"
            - name: OMPI_ALLOW_RUN_AS_ROOT_CONFIRM
              value: "1"
          resources:
            requests:
              cpu: "100m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"

```

CI/CD workflow摘要:

```

name: Build and deploy to Huawei CCE

on:
  push:
    branches:
      - main
      - master
  workflow_dispatch:

env:

```

```

FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true
SWR_REGISTRY: swr.cn-north-4.myhuaweicloud.com
SWR_NAMESPACE: cloud-compute-2026
BACKEND_IMAGE: backend
FRONTEND_IMAGE: frontend

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source
        uses: actions/checkout@v4

      - name: Set image tag
        run: echo "IMAGE_TAG=${GITHUB_SHA::7}" >> "$GITHUB_ENV"

      - name: Login to Huawei SWR
        uses: docker/login-action@v3
        with:
          registry: ${ env.SWR_REGISTRY }
          username: ${ secrets.SWR_USERNAME }
          password: ${ secrets.SWR_PASSWORD }

      - name: Build and push backend image
        run: |
          docker build -t "$SWR_REGISTRY/$SWR_NAMESPACE/$BACKEND_IMAGE:$IMAGE_TAG"
          ./backend
          docker push "$SWR_REGISTRY/$SWR_NAMESPACE/$BACKEND_IMAGE:$IMAGE_TAG"

      - name: Build and push frontend image
        run: |
          docker build -t "$SWR_REGISTRY/$SWR_NAMESPACE/$FRONTEND_IMAGE:$IMAGE_TAG"
          ./frontend
          docker push "$SWR_REGISTRY/$SWR_NAMESPACE/$FRONTEND_IMAGE:$IMAGE_TAG"

      - name: Configure kubectl
        run: |
          mkdir -p "$HOME/.kube"
          echo "${ secrets.CCE_KUBECONFIG_B64 }}" | base64 -d >
          "$HOME/.kube/config"
          kubectl version --client
          kubectl get nodes

      - name: Update CCE deployments
        run: |
          kubectl set image deployment/backend
          backend="$SWR_REGISTRY/$SWR_NAMESPACE/$BACKEND_IMAGE:$IMAGE_TAG"
          kubectl set image deployment/frontend
          frontend="$SWR_REGISTRY/$SWR_NAMESPACE/$FRONTEND_IMAGE:$IMAGE_TAG"
          kubectl rollout status deployment/backend --timeout=180s
          kubectl rollout status deployment/frontend --timeout=180s
          kubectl get deployment backend frontend -o wide

```

监控 YAML 摘要:

```

apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus-lite
rules:
  - apiGroups: [""]
    resources: ["nodes", "nodes/proxy", "services", "endpoints", "pods"]
    verbs: ["get", "list", "watch"]
  - nonResourceURLs: ["/metrics"]
    verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus-lite
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus-lite
subjects:
  - kind: ServiceAccount
    name: prometheus
    namespace: monitoring
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: monitoring
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s

    scrape_configs:
      - job_name: kubernetes-nodes-cadvisor
        scheme: https
        tls_config:
          insecure_skip_verify: true
        bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
        kubernetes_sd_configs:
          - role: node
        relabel_configs:
          - target_label: __address__
            replacement: kubernetes.default.svc:443
          - source_labels: [__meta_kubernetes_node_name]
            regex: (.+)

```

```

        target_label: __metrics_path__
        replacement: /api/v1/nodes/${1}/proxy/metrics/cadvisor

- job_name: kubernetes-pods
  kubernetes_sd_configs:
    - role: pod
  relabel_configs:
    - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
      action: keep
      regex: true
    - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
      action: replace
      target_label: __metrics_path__
      regex: (.+)
    - source_labels: [__address__,
      __meta_kubernetes_pod_annotation_prometheus_io_port]
      action: replace
      regex: ([^:]+)(?::\d+)?;(\d+)
      replacement: $1:$2
      target_label: __address__
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
    spec:
      serviceAccountName: prometheus
      containers:
        - name: prometheus
          image: prom/prometheus:v2.55.1
          args:
            - --config.file=/etc/prometheus/prometheus.yml
            - --storage.tsdb.path=/prometheus
            - --web.enable-lifecycle
          ports:
            - containerPort: 9090
          resources:
            requests:
              cpu: "100m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "768Mi"
          volumeMounts:
            - name: config
              mountPath: /etc/prometheus
      volumes:
        - name: config
          configMap:

```

```

        name: prometheus-config
---
apiVersion: v1
kind: Service
metadata:
  name: prometheus
  namespace: monitoring
spec:
  type: ClusterIP
  selector:
    app: prometheus
  ports:

```

边缘传感器发布脚本：

```

import json
import random
import time
from datetime import datetime, timezone

import paho.mqtt.client as mqtt

BROKER_HOST = "127.0.0.1"
BROKER_PORT = 1883
TOPIC = "edge/sensor/room-a"

def build_payload(seq):
    return {
        "device_id": "edge-room-a-01",
        "seq": seq,
        "temperature": round(24.0 + random.random() * 3.0, 2),
        "humidity": round(45.0 + random.random() * 10.0, 2),
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }

def main():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.connect(BROKER_HOST, BROKER_PORT, keepalive=30)
    for seq in range(1, 11):
        payload = build_payload(seq)
        client.publish(TOPIC, json.dumps(payload, ensure_ascii=False), qos=1)
        print(f"published {TOPIC} {payload}")
        time.sleep(0.5)
    client.disconnect()

if __name__ == "__main__":
    main()

```

边缘到云端 Redis 桥接脚本摘要：

```

import json
import os
import time

import paho.mqtt.client as mqtt
import redis

MQTT_HOST = os.getenv("MQTT_HOST", "127.0.0.1")
MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
MQTT_TOPIC = os.getenv("MQTT_TOPIC", "edge/sensor/#")

REDIS_HOST = os.getenv("REDIS_HOST", "127.0.0.1")
REDIS_PORT = int(os.getenv("REDIS_PORT", "63790"))
REDIS_PASSWORD = os.getenv("REDIS_PASSWORD") or None

rdb = redis.Redis(
    host=REDIS_HOST,
    port=REDIS_PORT,
    password=REDIS_PASSWORD,
    decode_responses=True,
)

def on_connect(client, userdata, flags, reason_code, properties):
    print(f"connected mqtt reason={reason_code}")
    client.subscribe(MQTT_TOPIC, qos=1)

def on_message(client, userdata, message):
    payload = message.payload.decode("utf-8")
    data = json.loads(payload)
    rdb.lpush("edge:sensor:latest", json.dumps(data, ensure_ascii=False))
    rdb.ltrim("edge:sensor:latest", 0, 19)
    rdb.hset("edge:sensor:last", mapping={
        "topic": message.topic,
        "device_id": data.get("device_id", ""),
        "seq": data.get("seq", 0),
        "temperature": data.get("temperature", 0),
        "humidity": data.get("humidity", 0),
        "timestamp": data.get("timestamp", ""),
    })
    print(f"stored topic={message.topic} seq={data.get('seq')}")

def main():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(MQTT_HOST, MQTT_PORT, keepalive=30)
    client.loop_start()
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        pass
    finally:

```

```
client.loop_stop()
client.disconnect()

if __name__ == "__main__":
    main()
```

六、总结

本次课程设计把容器化、Kubernetes 编排、云负载均衡、持久化存储、配置分离、弹性伸缩和并行计算串成了一套完整流程。第一部分中，镜像架构、SWR 鉴权、ELB 注解、ConfigMap 挂载和 PVC 持久化都需要逐项验证，单看 YAML 是否写完并不能证明系统可用。实际排查时，`kubectl describe pod`、`kubectl describe svc`、`kubectl logs` 和 `kubectl top` 比单纯看 `get pods` 更有价值。

MPI 部分让我更直观看到云原生环境下并行政程序的额外成本。数值积分算法本身容易并行，但容器启动、SSH 建连、跨节点通信和 Reduce 同步都会影响实测加速比，所以实测结果不会简单等于理论线性加速。非阻塞通信在本实验中有小幅改善，但由于通信数据量只有一个局部和，优化空间有限。监控、CI/CD 和 MQTT 加分项进一步说明，一个云应用不仅要能部署，还要能观测、能自动交付，并能和边缘侧数据流连接起来。

附录：文件清单

- 后端：backend/app.py、backend/Dockerfile、backend/requirements.txt
- 前端：frontend/static/index.html、frontend/nginx.conf、frontend/Dockerfile
- Kubernetes：k8s/00-config.yaml 至 k8s/05-hpa.yaml
- MPI：mpi/integral_serial.py、mpi/integral_mpi.py、mpi/integral_mpi_nonblocking.py、mpi/mpijob.yaml
- 监控：k8s/bonus-monitoring/monitoring.yaml
- CI/CD：.github/workflows/deploy-cce.yml
- 边缘 MQTT：bonus/edge-mqtt/
- 证据文件：report/evidence/
- 报告截图：report/final-assets/